ECC P-256 & SHA-256 — THE COMPLETE WORKING EXPLAINER

What ECC and P-256 actually mean, and step by step, exactly how the SHA hash system works together with ECC-P256 to sign and verify a ZerôPâ■ Money transaction

Prepared for Tom Ugbodaga, Shareholder & CEO. Draft: August 14, 2026. Plain-language technical reference — independent of Base44, stored on GitHub.

1. The Full Meanings, Plainly

ECC = Elliptic Curve Cryptography

A family of cryptographic techniques built on the mathematics of elliptic curves — not the vehicle kind of curve, a specific type of mathematical curve with a special property: you can define an 'addition' operation between points sitting on the curve, and that operation is very easy to do forwards, but computationally infeasible to undo backwards. That one-way property is the entire foundation of why ECC is secure.

P-256 = the specific curve we use

P-256 (also called secp256r1 or prime256v1 in different software) is one specific, standardized elliptic curve published by the U.S. National Institute of Standards and Technology (NIST). The '256' refers to the size of the numbers involved — 256 bits. It is one of the most widely trusted, widely implemented curves in the world, used in TLS/HTTPS website security, government systems (FIPS 140-2 approved), and — critically for us — it is small and fast enough to run on a cheap embedded secure element chip, which is exactly why it fits a $44 offline payment terminal instead of needing a full server.

SHA = Secure Hash Algorithm

A family of one-way 'fingerprinting' functions. SHA-256 (the specific version paired with P-256) takes any input of any size — a word, a sentence, an entire file — and produces a fixed-size, 256-bit (32-byte) output called a hash or digest. The same input always produces the exact same hash. Changing even a single character of the input produces a completely different, unpredictable hash. You cannot work backwards from the hash to recover the original input.

ECDSA = Elliptic Curve Digital Signature Algorithm

The actual named algorithm that combines SHA-256 and the P-256 curve together to produce a digital signature. When people say 'ECC P-256 signing,' they specifically mean ECDSA using the P-256 curve and the SHA-256 hash function. This is the exact mechanism running inside every ZerôPâ■ Money terminal's secure element every time a tap is signed.

---

2. The Plain-English Analogy First — A Wax Seal

Before the math: imagine a medieval letter sealed with wax and a signet ring.

- Your signet ring (the private key) is the one thing only you physically possess, and it never leaves your hand.
- The unique impression pattern it leaves in wax (the public key) can be shown to absolutely anyone — showing it to people doesn't let them make their own ring.
- Before sealing, the letter's exact content gets condensed into a unique 'fingerprint' of that specific message (the SHA-256 hash) — like taking a precise measurement of exactly what's being sealed.
- Pressing the ring into wax over that fingerprint (signing) produces a seal that could only have come from your ring, proving both who sent it and that the letter's content hasn't been altered since.
- Anyone receiving the letter can check the seal pattern against your known ring pattern (verification) without ever touching or needing your actual ring.

Everything below is simply the precise mathematical version of that same idea.

---

3. Step 1 — Where the Keys Come From (Happens Once, at Manufacturing)

1. The secure element chip picks a genuinely random number, d, somewhere between 1 and a huge number called the curve order (n) — this random number IS the private key. It never leaves the chip, ever, not even to us as manufacturer.
2. The P-256 curve has one universally agreed-upon fixed starting point on it, called the generator point, G — this is public, defined by the standard, identical on every P-256 device in the world.
3. The chip computes a new point on the curve: Q = d × G — meaning 'add G to itself, d times' using the curve's special addition rule. This resulting point, Q, is the public key.
4. Going from d to Q (multiply forward) is fast and easy. Going from Q back to d (the 'elliptic curve discrete logarithm problem') is, with a 256-bit curve, computationally infeasible with any current or foreseeable technology — this one-way gap is the entire security guarantee.

---

4. Step 2 — Signing a Transaction (Happens at Every Tap)

Example message being signed at a real tap: “account X, new balance = old balance minus amount, counter = previous counter + 1.”

1. Hash the message: run the exact transaction data through SHA-256, producing a fixed 256-bit digest, call it h. Two different transaction amounts will always produce two totally different, unpredictable digests.
2. Generate a fresh random number, k, used only for this one signature and never reused (reusing k across two signatures would leak the private key — this is a real, well-known implementation bug found in some past inline systems, which is exactly why correct P-256 implementations always use a fresh random or deterministic-per-message k).
3. Compute a new curve point R = k × G, then take R's x-coordinate and reduce it modulo n (the curve order) — call this value r. This becomes the first half of the signature.
4. Compute the second half: s = k■¹ × (h + r × d) mod n — combining the message hash (h), the value from step 3 (r), the private key (d), and the inverse of the random k, all under modular arithmetic.
5. The finished digital signature is the pair (r, s) — two numbers, roughly 32 bytes each, 64 bytes total. This is what gets attached to and travels with the transaction.
6. Critically: the private key, d, was used inside this calculation but never leaves the chip and is never transmitted anywhere — only the output numbers (r, s) travel with the transaction.

---

5. Step 3 — Verifying a Signature (Happens at the Terminal, or Later at the Hub)

The verifier has: the original message, the signature (r, s), and the signer's public key, Q. The verifier does NOT have and does NOT need the private key.

1. Hash the received message the exact same way: run it through SHA-256 to get h — the same digest the signer would have produced, IF the message hasn't been altered.
2. Using r, s, h, and the public key Q, the verifier runs a defined sequence of elliptic curve point additions and modular arithmetic operations (the reverse-shaped counterpart of the signing math) that produces a new point on the curve.
3. Take that resulting point's x-coordinate and reduce it modulo n.
4. Compare that result to r, the first half of the signature that was sent. If they match exactly: the signature is mathematically valid — accept the transaction. If they don't match, even by one bit: reject immediately.
5. What a valid match actually proves: this exact message could only have been signed by whoever holds the private key matching public key Q, AND the message was not altered in transit — both facts proven simultaneously by one successful check.

---

6. Why Hash With SHA-256 First, Instead of Signing the Raw Message Directly

- Fixed size, regardless of message length: the signing math (step 4 above) needs a number of a specific, consistent size to plug into the equation — a hash always gives exactly 256 bits, whether the original message was 10 characters or 10,000.
- Speed: hashing a long message is far faster than running the actual elliptic curve signing math directly against long raw data.
- Security: signing a hash instead of raw data closes off certain mathematical attacks that become possible if an attacker can choose or influence the exact raw content being signed directly.

---

7. Exactly Where This Runs Inside a Real ZerôPâ■ Money Tap

This is Frame 7 of the tap sequence described in the founder double-spend analysis, made precise:

1. The customer's chip has already computed its new balance and incremented its own hardware counter, entirely internally (Frames 5-6).
2. That new balance + counter value becomes the 'message' — it gets hashed with SHA-256, then signed using ECDSA with the chip's own P-256 private key, exactly as described in Sections 4 above.
3. The resulting signature (r, s) travels with the transaction token to the receiving terminal.
4. The receiving terminal already has (or can be given) the customer chip's public key, and runs the verification steps in Section 5. A valid match means: this decrement really was produced by this exact, unclonable chip, and the balance/counter values weren't tampered with in transit.
5. This is also precisely why cloning fails and rollback fails (as covered in the frame-by-frame document): even if an attacker perfectly copied the balance and counter numbers, they cannot produce a valid (r, s) signature over those numbers without the private key, d — which never leaves the original chip and cannot be extracted, only replicated by physically reproducing the exact silicon-level randomness the PUF is built on.

---

8. Quick Glossary

Term	Plain-English Meaning
Private key (d)	A secret random number generated once inside the chip. Never leaves the chip. Proves identity when used to sign.
Public key (Q)	A point on the curve derived from the private key. Safe to share with anyone; used only to verify, never to sign.
Generator point (G)	A single fixed, publicly-known starting point on the P-256 curve, identical on every device using this standard.
Curve order (n)	A huge fixed number that all the 'modulo' arithmetic wraps around — part of the P-256 standard, same for everyone.
Hash / digest (h)	A fixed-size 256-bit fingerprint of a message, produced by SHA-256. Same input always gives the same hash; can't be reversed.
Signature (r, s)	Two numbers produced by the signing process. Travels with the transaction. Proves authorship and integrity together.
ECDSA	The named algorithm combining SHA-256 hashing + P-256 curve math to produce and check signatures — what actually runs on our chip.
Modulo (mod)	Ordinary 'remainder after division' arithmetic — e.g. 17 mod 5 = 2. Used throughout to keep all numbers within a fixed range.

Nothing in this document is proprietary to ZerôPâ■ Money — ECC P-256 and SHA-256 are open, public, internationally standardized building blocks used by banks, browsers, and governments worldwide. What IS proprietary is HOW we've

combined this standard signing mechanism with PUF-based unclonable hardware and distance-bounding into a specific stored-value architecture — covered separately in the founder frame-by-frame document.

Tom Ugbodaga Shareholder & CEO, BETONIQ WEST LTD